



Arhitectură ML distribuită: Azure Storage + Databricks Services

Conf.dr. Cristian KEVORCHIAN
Facultatea de Matematică și Informatică

Apache SPARK



Apache Spark(initial a apartinut Berkeley) este un **motor de procesare distribuit**, in-memory, conceput pentru scalabilitate și performanță în Big Data.



Azure Databricks extinde Spark într-un mediu de cloud full managed, oferind o platformă PaaS pentru AI, ML și ingineria datelor.

Poziționare Conceptuală: Apache Spark vs Azure Service Fabric

	Apache Spark	Azure Service Fabric
Tip de sistem	Motor de procesare distribuit	Platformă de orchestrare microservicii
Nivel de abstractizare	Mediu de execuție pentru joburi de date	Mediu de execuție pentru aplicații stateful/stateless
Scop principal	Procesare paralelă de date (batch, streaming, ML)	Orchestrare, scalare și reziliență pentru microservicii
Model de programare	RDD(Resilient Distributed DataSet) / DataFrame / SQL / MLib	Actori / servicii / containere
Persistență	Delta Lake / Parquet / Hive	State management intern sau externalizat (Azure Storage)
Execuție	DAG Scheduler, fault tolerance, cluster manager	Service lifecycle, partitioning, failover
Integrare în Azure	Azure Databricks (PaaS)	Azure Service Fabric (IaaS/PaaS)

Contextul Spark

1. Începutul (Spark Context)

- **Contextul Istoric:** La început, Spark se concentra pe **prelucrarea rapidă, in-memory**, înlocuind modelul lent **MapReduce**.
- **Invenția: Spark Context** a fost singurul punct de intrare. Rolul său era strict de a gestiona resursele clusterului și de a opera cu o abstractizare low-level a datelor: **RDD-urile (Resilient Distributed Datasets)**.
- **Limitare:** Lucrul cu RDD-uri era *low-level* și necesita mult cod boilerplate pentru operațiunile simple.

2. Necesitatea Datelor Structurate (SQL Context)

- **Contextul Istoric:** Comunitatea Big Data avea nevoie de un mod mai ușor și mai familiar de a lucra cu datele, similar cu bazele de date tradiționale.
- **Invenția:** A fost introdus **SQL Context** (construit peste Spark Context). Acesta a adus abstractizarea **DataFrame** (date structurate în tabele și coloane) și a permis utilizatorilor să scrie interogări direct în **SQL**.
- **Impact:** DataFrame-urile și SQL Context-ul au permis introducerea **Catalyst Optimizer**, care analiza interogările și le optimiza automat, sporind drastic performanța.

3. Integrarea Ecosistemului (Hive Context)

- **Contextul Istoric:** Multe organizații mari foloseau deja **Apache Hive** pentru a stoca schemele și metadatele datelor lor (**Hive Metastore**).
- **Invenția:** A fost introdus **Hive Context**, ca o extensie sau o specializare a SQL Context-ului. Rolul său a fost de a asigura **compatibilitatea retroactivă** completă cu Hive.
- **Funcționalitate:** Îi permitea Spark-ului să acceseze schemele tabelor stocate în Metastore și să ruleze cod scris în **HiveQL**, facilitând migrarea de la Hive la Spark.

Funcționalitate	Descriere
Rol principal	Punct de intrare unificat în Spark
Include	SparkContext, SQLContext, HiveContext, Catalog
Tipuri de date suportate	DataFrame, Dataset, RDD
Optimizări	Catalyst (planificare) + Tungsten (execuție)
Persistență	Poate salva în Parquet, Delta, Hive, JDBC etc.
Interfață SQL	Da – suport complet pentru SQL
Acces la SparkContext	spark.sparkContext
Creare explicită	SparkSession.builder.getOrCreate()
Utilizare tipică	ETL, machine learning, SQL, streaming, analiza de date

SparkSession

SparkSession este abstracția de top în Apache Spark, introdusă în versiunea 2.x, care unifică toate punctele de intrare în Spark: SQL, DataFrame, Dataset, Hive, Streaming și MLlib. Este interfața principală prin care utilizatorii interacționează cu ecosistemul Spark.

Arhitectura datelor în Spark

RDD(Resilient Distributed DataSet) – este o abstracție fundamentală în Apache Spark care reprezintă o colecție distribuită, imuabilă și tolerantă la erori de elemente, ce pot fi procesate în paralel pe un cluster(Matei Zaharia, 2012).

- Nu are schema — doar obiecte.
- Toate celelalte abstracții (DataFrame, Dataset) sunt construite peste RDD-uri.

Proprietate	Descriere
Distribuit	Datele sunt împărțite în partiții și procesate pe mai multe noduri.
Imuabil	Odată creat, un RDD nu poate fi modificat — doar transformat în altul.
Tolerant la erori	Folosește lineage (istoricul transformărilor) pentru a reconstrui datele pierdute.
Evaluare leneșă	Operațiile nu sunt executate imediat, ci doar când se aplică o acțiune.
Operații	<code>transformations</code> (ex: <code>map</code> , <code>filter</code>) și <code>actions</code> (ex: <code>collect</code> , <code>count</code>)

Tabel comparativ între **RDD**, **DataFrame** și **Dataset**

	RDD	DataFrame	Dataset
Schema	Nu are schemă	Are schemă	Are schemă
Structură	Obiecte distribuite	Tabel	Obiecte tipizate distribuite
Operații	Funcționale (map, filter)	Declarative (SQL)	Funcționale și declarative

Fault toleranță Azure Service Fabric

- **Service Fabric** este o platformă de orchestrare pentru microservicii și aplicații distribuite, cu suport pentru **stateful** și **stateless services**.
- **Mecanisme de fault tolerance:**
 - **Replicație automată:** pentru serviciile stateful, datele sunt replicate între noduri (Primary + Secondaries).
 - **Failover rapid:** dacă un nod cade, Service Fabric promovează automat o replică secundară pentru rolul de primară.
 - **Health monitoring:** monitorizează starea serviciilor și nodurilor, cu intervenții automate.
 - **Upgrade-uri rolling:** permite actualizări fără downtime, cu rollback automat în caz de eșec.
 - **Placement constraints:** evită plasarea replicilor pe aceleași noduri sau fault domains.
- **Avantaje:**
 - Granularitate mare în controlul fault domains și update domains.
 - Suport pentru aplicații cu stare (stateful) cu replicare și consistență garantată.
 - Integrare nativă cu Azure pentru scalare și disponibilitate.

Fault Tolerance

-Apache Spark

RDD

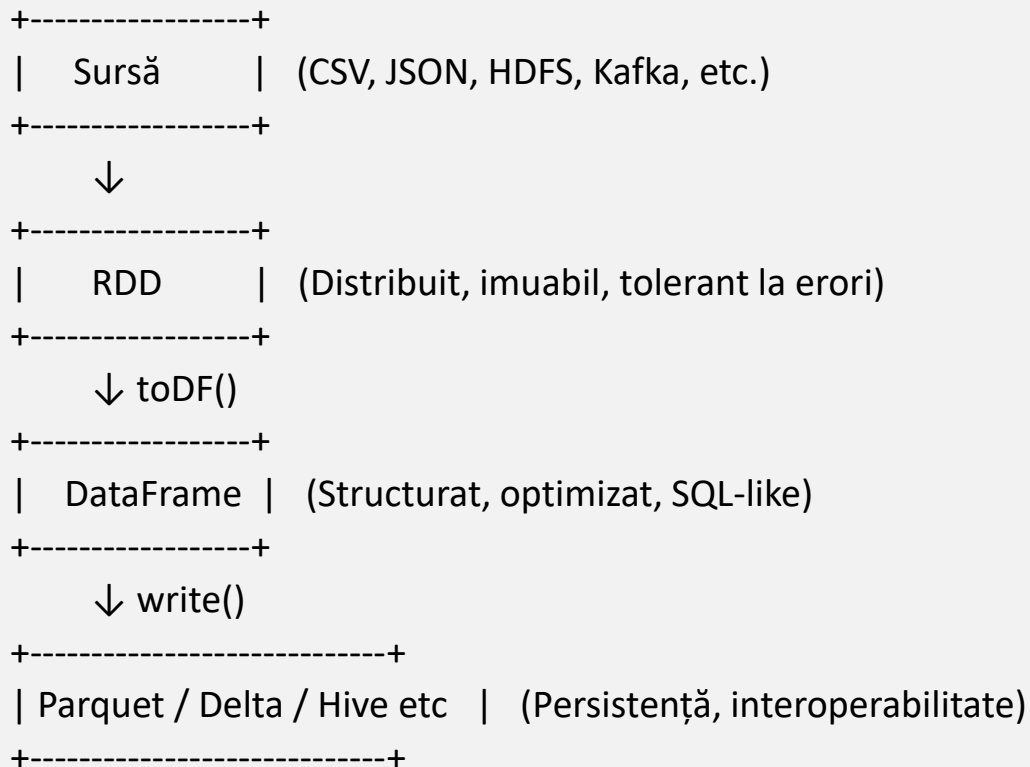
- **Resilient Distributed Dataset (RDD)** este structura de bază în Spark pentru procesarea distribuită a datelor.
- **Mecanisme de fault tolerance:**
 - **Lineage-based recomputation:** dacă o partiție RDD este pierdută, Spark o poate reconstrui folosind istoricul transformărilor (lineage).
 - **Immutable data:** RDD-urile sunt imutabile, ceea ce permite refacerea lor în mod determinist.
 - **Task retry:** dacă un task eșuează, Spark îl reexecută automat pe alt executor.
- **Checkpointing** (optional): pentru RDD-uri complexe, se pot salva explicit în stocare distribuită (HDFS, ADLS) pentru a evita recalculări costisitoare.
- **Avantaje:**
 - Fault tolerance implicită prin designul RDD.
 - Scalabilitate masivă pentru procesarea datelor.
 - Nu necesită replicare activă – se bazează pe recalculare.

Comparatie ASF vs RDD

	Azure Service Fabric	Spark RDD
Tip sistem	Orchestrator microservicii	Motor de procesare date
Fault tolerance	Replicare + failover	Recomputation + retry
Persistență stare	Da (stateful services)	Nu (RDD este temporar, opțional checkpoint)
Recuperare în caz de eșec	Promovare replică secundară	Recalculare partiții pierdute
Granularitate control	Înaltă (noduri, domenii)	Limitată (la nivel de task/partiție)
Tipuri de aplicații	Aplicații distribuite, microservicii	Procesare big data, batch/streaming

Fluxul de date în SPARK

—



Fie secvența de operații:

-
- Încărcarea datelor din Hive Metastore (default.date_regresie)
 - Aplicarea regresiei nelineare $f: y = f(u, v, w)$
 - Aplicarea regresiei liniare $g: y = g(q)$
 - Calculul diferenței $h = f - g$
 - Salvarea coeficienților în Delta Lake
 - Exportul în ADLS Gen2
 - Vizualizarea funcției h

Fundamente teoretice



- Regresia polinomială extinde regresia liniară prin includerea termenilor de ordin superior
- f : model nelinear cu PolynomialExpansion (grad 3)
- g : model liniar simplu pe variabila q
- $h = f - g$: diferență între predicțiile celor două modele
- Coeficienții modelelor oferă interpretabilitate asupra influenței fiecărei variabile

Implementare în Databricks

-
- Datele sunt încărcate din tabelul Hive Metastore: default.date_regresie
 - Se aplică VectorAssembler și PolynomialExpansion
 - Se antrenează două modele LinearRegression
 - Se calculează $h = y_f - y_g$
 - Se extrag coeficienții și se salvează în Delta Lake
 - Se exportă în ADLS Gen2 folosind protocolul abfss://

Rezultate și vizualizare

-
- Coeficienții modelelor f , g și h au fost salvați în format Delta
 - Fișierul a fost exportat în ADLS Gen2 în containerul date2abfs
 - Funcția $h = f - g$ a fost reprezentată grafic cu Plotly
 - Graficul oferă o perspectivă asupra diferenței dintre modelele nelinear și liniar

Concluzii

-
- Fluxul implementat permite analiză avansată și trasabilă
 - Databricks oferă integrare nativă cu ADLS și Delta Lake
 - Modelele f și g pot fi extinse cu evaluări suplimentare (R^2 , MSE)
 - Vizualizarea funcției h ajută la înțelegerea diferențelor de comportament

MLflow în Azure Databricks

- MLflow este o platformă open-source pentru managementul ciclului de viață al modelelor ML.
- Aplicație:
 - Tracking automat al experimentelor f, g și h
 - Salvarea hiperparametrilor și metadatelor
 - Compararea performanței modelelor
 - Reproducibilitate și versionare

Integrare MLflow în codul Databricks

- Exemplu de cod pentru logarea unui model:

```
import mlflow
with mlflow.start_run():
    model = LinearRegression().fit(X, y)
    mlflow.sklearn.log_model(model, "model_f")
    mlflow.log_metric("rmse", rmse_val)
```

- Toate experimentele pot fi vizualizate în UI-ul MLflow din Databricks.

Text2SQL în Databricks

- **Databricks Assistant AI/BI Genie**
- Databricks Assistant este un AI conversațional integrat direct în interfața Databricks, care permite:
 - Generarea de cod SQL din întrebări în limbaj natural.
 - Autocompletare inteligentă.
 - Debugging automat și sugestii de corectare.
 - Execuție serverless a codului.
 - Citări și sugestii din documentație oficială.

Exemplu în **AIBI Genie**

- SQL pentru schema mytaxi.trips:

Selectează ultimele 1000 de curse (după data și ora de preluare)

Calculează suma tarifelor și distanța totală, grupate după tipul de plată